

System Analysis Documentation (SAD)

BUILDORA AI: *AI-Powered Construction Workflow Automation*

UM HACKATHON 2026 - Domain 1: AI Agentic Workflow Automation

TEAM NAME

Dang Wangi

TEAM MEMBERS

Nur Alia Syahira Binti Nor Azam

Muhammad Khaidhir Bin Mohd Razin

Abdul Hafiz Bin Mohd Noor Azman

Nur Farah Binti Ahmad Nazri

Fakhrul Azim Bin Ahmed Mardzukie

1. Introduction

Buildora AI is an agentic workflow automation platform designed to solve the critical inefficiencies in the Malaysian construction industry. By leveraging a multi-agent orchestration system, Buildora automates document parsing, project monitoring, and CIDB permit compliance.

1.1 Purpose

The System Analysis Document (SAD) outlines the technical architecture, data flows, and design decisions behind Buildora AI. It serves as a blueprint for the development and QA teams to ensure the system meets the requirements defined in PRD v1.0.

Key elements covered:

1. **Architecture:** A stateful multi-agent system using LangGraph and GLM via [Z.AI](#) and OpenAI.
2. **Data Flows:** The movement of data from document upload through agent processing to Firebase/Redis persistence and final report generation.
3. **Model Process:** End-to-end workflows for document ingestion, delay detection, and permit validation.
4. **Role of Reference:** A technical guide for developers and stakeholders to understand the system's high-level design.

1.2 Background

Malaysian construction projects face significant losses (RM 5.75B–9.5B annually) due to manual tracking and uncoordinated permit submissions. Buildora replaces fragmented WhatsApp/Excel workflows with a centralized, AI-driven “Single Source of Truth.”

1.3 Target Stakeholders

Stakeholder	Role	Expectations
Project Manager (PM)	Uploads docs, monitors dashboard, receives alerts.	Automated tracking, early warnings, and zero-effort reporting.
Site Supervisor	Submits milestone updates via mobile web.	Simple, quick form-based updates without paperwork.
Compliance Officer	Reviews CIDB compliance scores and gap lists.	Accurate validation against CIDB BISQ before submission.
Development Team	Builds and maintains the platform.	Modular architecture, clear API contracts, and stateful orchestration.
QA Team	Validates system behavior and AI accuracy.	Defined test cases, clear request/response formats, and edge case coverage.

2. System Architecture & Design

2.1 High-Level Architecture

Type	Details
System	Web Application (Responsive React + Vite)
Architecture	Orchestrator-driven Multi-Agent System (LangGraph)

Buildora is structured as a client-server system. The **React Frontend** communicates with a **FastAPI Backend**, which triggers a **LangGraph Orchestrator**. The orchestrator manages five specialized agents that share state via **Firebase Realtime Database** (or Redis for low-latency state).

2.2 LLM as Service Layer (GLM via Z.AI)

GLM serves as the core reasoning engine across all agents:

- **Agent A:** Uses GLM for structured field extraction from parsed text/OCR.
- **Agent C:** Uses GLM to cross-reference project state against the CIDB BISQ knowledge base.
- **Agent E:** Uses GLM for multi-lingual summarization (BM/English) in reports.
- **Context Handling:** Documents >4,000 tokens are chunked by page; prompts use few-shot examples for extraction accuracy.

2.3 Technological Stack

- **Frontend:** React + Vite + Tailwind CSS (Responsive Web).
- **Backend:** FastAPI (Python 3.11+) for async agent orchestration.
- **AI Engine:** GLM-4-Flash via Z.AI and OpenAI API.
- **Orchestration:** LangGraph for stateful multi-agent workflows.
- **Database/State:** Firebase Realtime Database (Shared State) + Firestore (Audit Logs).
- **File Storage:** Firebase Storage (PDFs/Images).
- **Alerts:** Telegram Bot API.

2.4 Key Data Flows

2.4.1 Data Flow Diagram (DFD)

1. **Input:** PM uploads PDF -> FastAPI -> Firebase Storage.
2. **Process A:** Agent A parses PDF -> GLM extracts fields -> Writes to Firebase Realtime DB.
3. **Process B:** Agent B reads DB -> Compares milestones to baseline -> Flags delays.
4. **Process C:** Agent C reads DB + CIDB JSON -> GLM validates compliance -> Writes score to DB.
5. **Output:** Agent D sends Telegram alert; Agent E generates PDF/XLSX from DB state.
6. **ChatBot Panel:** User input → knowledge base → POST /chat (FastAPI)

2.4.2 Database Schema (Logical)

- **Projects:** project_id, contractor_name, contract_value, start_date, end_date.
- **Milestones:** milestone_id, project_id, baseline_date, actual_date, status.
- **Compliance:** project_id, score, gap_list (JSON), last_checked.
- **Knowledge Base:** cidb_rules, compliance_rules, doc_checklist, document_definitions.

3. Functional Requirements & Scope

3.1 Minimum Viable Product (MVP)

#	Feature	Description
1	AI Document Ingestion	Automated extraction of 15+ fields from construction contracts.
2	Stateful Monitoring	Real-time detection of >3 day delays or >8% cost variance.
3	CIDB Compliance Check	Automated scoring against the BISQ checklist (Conditional).
4	Automated Reporting	Branded PDF and XLSX generation in <10 seconds.

3.2 Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Performance	Full pipeline (Upload to PDF) < 15s.	Async processing and pre-cached GLM responses.
Reliability	99% state persistence.	Firebase Realtime DB with persistent listeners.
Token Latency	GLM extraction < 6s per call.	Prompt optimization and response caching.
Cost Efficiency	Minimize token usage.	Chunking large docs and fixed-length extraction prompts.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

- **Priority Matrix:** P0 (Critical) for document extraction and delay alerts. If Agent A accuracy <70%, manual review is triggered.
- **Emergency Rollback:** Use of Docker images for instant rollback to “Golden Version” if the LangGraph orchestrator fails.

4.2 Assumptions

- Users have a stable internet for document uploads.
- Documents are in standard PDF/DOCX or clear image formats.
- GLM-4-Flash provides sufficient accuracy for Malaysian construction terminology.

4.3 External Dependencies

Tool	Purpose	Risk
Z.AI GLM API	Core AI reasoning engine.	High (Rate limits/Latency). Mitigation: Caching.
Firebase	Shared state and file storage.	Medium (Connectivity). Mitigation: Local state sync.
Telegram API	Real-time alerting.	Low (Service downtime).

5. Project Management & Team Contributions

- **Timeline:** 10-day build window.
 - Day 1-3: Architecture & Agent A;
 - Day 4-7: Agent B & C;
 - Day 8-9: Agent D & E;
 - Day 10: Testing & Demo Prep.
- **Team Roles:**
 - Nur Alia Syahira (PM/Logic),
 - Fakhrul Azim (AI/Backend),
 - Nur Farah (Frontend/Reports),
 - Abdul Hafiz (CIDB/Alerts),
 - Muhammad Khaidhir (Database/Monitoring).